

EXPERIMENT 2

DEMONSTRATION OF STAGES OF COMPILE AND EXECUTION OF A C PROGRAM

Theory

A C program does not directly convert from source code into an executable program in a single step. It passes through different stages before execution.

The major stages are:

1. **Preprocessing**
2. **Compilation**
3. **Assembly**
4. **Linking**
5. **Execution**

The GCC option `--save-temps` is used to preserve the intermediate files generated during these stages.

The general compilation process is:

Source Program (.c) → Preprocessed File (.i) → Assembly File (.s) → Object File (.o) → Executable File → Execution

Program

The following C program was written to calculate the area of a circle.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    float radius, area;
```

```
    printf("Enter the radius of the circle: ");
```

```
    scanf("%f", &radius);
```

EXPERIMENT 2

DEMONSTRATION OF STAGES OF COMPILE AND EXECUTION OF A C PROGRAM

```
area = 3.14 * radius * radius;
```

```
printf("Area of the circle = %.2f\n", area);
```

```
return 0;
```

```
}
```

The formula used is:

Area of Circle = $\pi \times r \times r$

where r represents the radius of the circle.

Procedure

Step 1: Create a directory for Experiment 2

The following commands were used:

```
mkdir experiment2
```

```
cd experiment2
```

The present working directory was verified using:

```
pwd
```

The directory was:

```
/home/kali/experiment2
```

Step 2: Creation of C Source Program

The source program was created using the vi editor.

Command:

EXPERIMENT 2

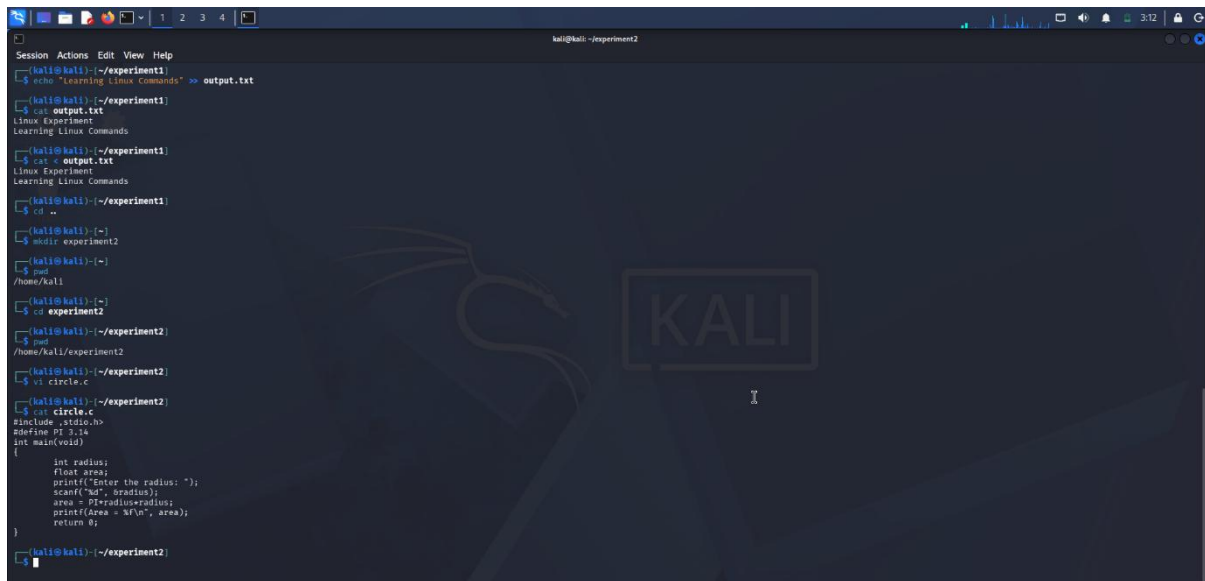
DEMONSTRATION OF STAGES OF COMPIATION AND EXECUTION OF A C PROGRAM

vi circle.c

The program for calculating the area of a circle was entered and saved as circle.c.

The contents of the program were verified using:

cat circle.c



```
Session Actions Edit View Help
(kali@kali)-[~/experiment1]
$ echo "Learning Linux Commands" >> output.txt
(kali@kali)-[~/experiment1]
$ cat output.txt
Learning Linux Commands
(kali@kali)-[~/experiment1]
$ cat < output.txt
Learning Linux Commands
(kali@kali)-[~/experiment1]
$ cd ..
(kali@kali)-[~]
$ mkdir experiment2
(kali@kali)-[~]
$ cd experiment2
(kali@kali)-[~/experiment2]
$ vi circle.c
(kali@kali)-[~/experiment2]
$ cat circle.c
#include <stdio.h>
#define PI 3.14
int main(void)
{
    int radius;
    float area;
    printf("Enter the radius: ");
    scanf("%d", &radius);
    area = PI*radius*radius;
    printf("Area = %f\n", area);
    return 0;
}
```

Figure 1: C source program for calculation of area of a circle

Stages of Compilation

1. Source Program

The original C program is stored in the file:

circle.c

This file contains the C instructions written by the programmer.

2. Compilation using --save-temps

The program was compiled using the following command:

```
gcc --save-temps circle.c -o circle
```

EXPERIMENT 2

DEMONSTRATION OF STAGES OF COMPILE AND EXECUTION OF A C PROGRAM

The `--save-temps` option instructs GCC to preserve the intermediate files produced during the compilation process.

After compilation, the following command was executed:

`ls`

The following files were generated:

`circle.c`

`circle.i`

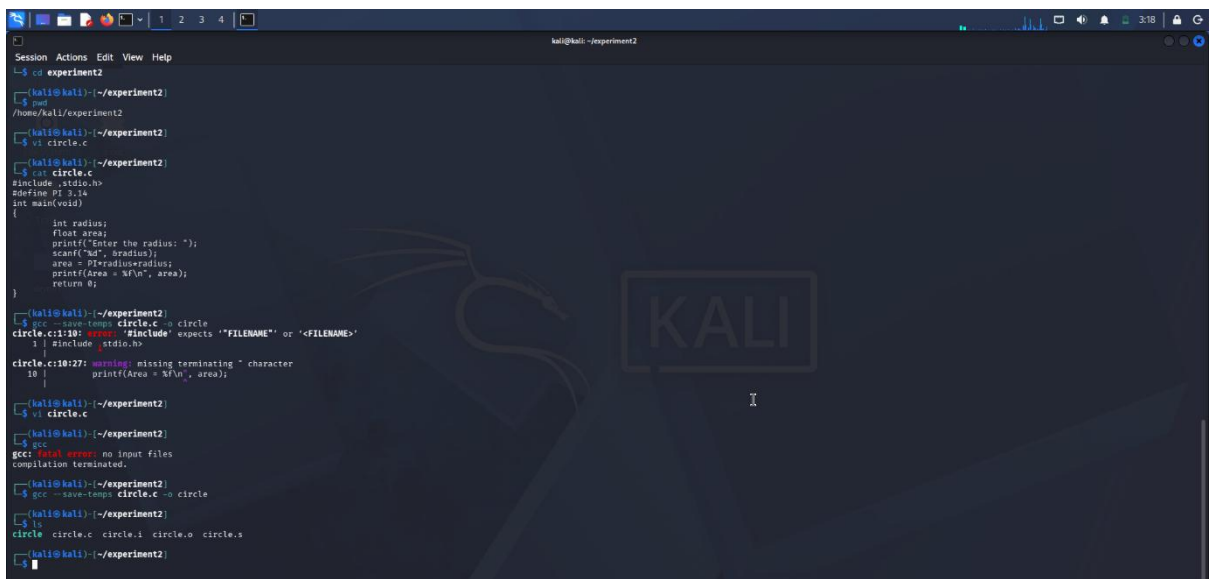
`circle.s`

`circle.o`

`circle`

Their purposes are:

- `circle.c` – Original C source program
- `circle.i` – Preprocessed source program
- `circle.s` – Assembly language program
- `circle.o` – Object file
- `circle` – Final executable file



```
Session Actions Edit View Help
~$ cd experiment2
~$ ls
~/home/kali/experiment2
~$ vi circle.c
~$ cat circle.c
#include <stdio.h>
#define PI 3.14
int main(void)
{
    int radius;
    float area;
    printf("Enter the radius: ");
    scanf("%d", &radius);
    area = PI*radius*radius;
    printf("Area = %f\n", area);
    return 0;
}
~$ gcc --save-temps circle.c -o circle
circle.c:18:27: error: 'include' expects 'FILENAME' or '<FILENAME>'
 18 | #include <stdio.h>
    |         ^
circle.c:18:27: warning: missing terminating ' character
 18 | #include <stdio.h>
    |         ^
~$ gcc
gcc: fatal error: no input files
compilation terminated.
~$ gcc --save-temps circle.c -o circle
~$ ls
circle.c circle.i circle.o circle.s
~$
```

EXPERIMENT 2

DEMONSTRATION OF STAGES OF COMPILETIME AND EXECUTION OF A C PROGRAM

Figure 2: Intermediate files generated using GCC --save-temps

3. Preprocessing Stage

During preprocessing, the C preprocessor processes preprocessor directives such as:

`#include <stdio.h>`

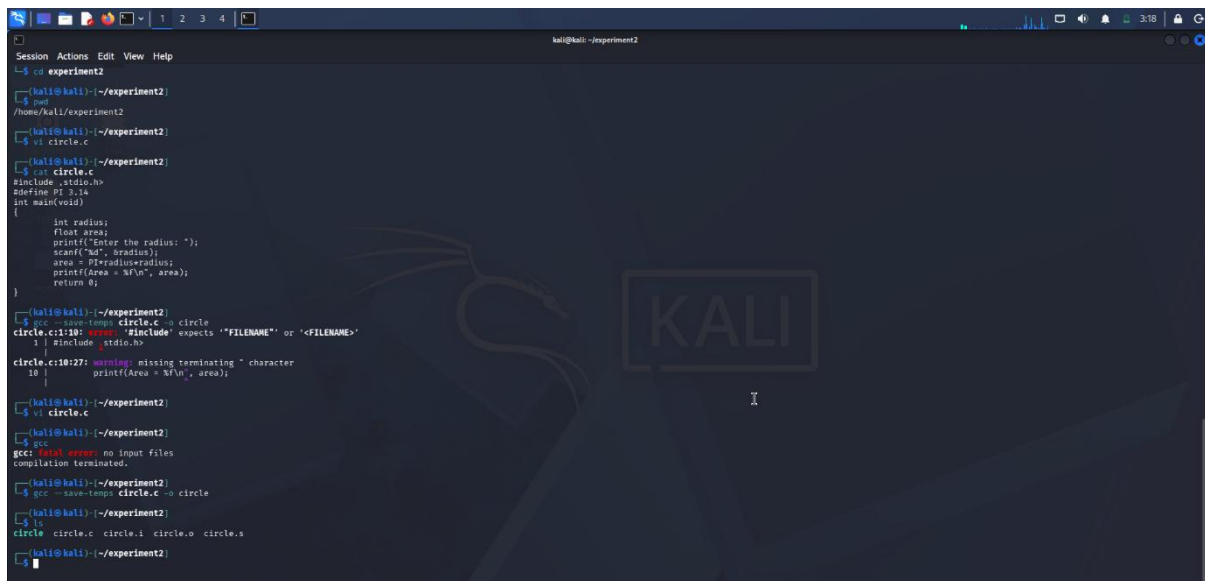
The output of this stage is stored in:

`circle.i`

A portion of the preprocessed file was viewed using:

`head circle.i`

The `.i` file contains the expanded source program after preprocessing.



```
Session Actions Edit View Help
kali@kali: ~/experiment2
$ cd experiment2
$ pwd
/home/kali/experiment2
$ vi circle.c
$ cat circle.c
#include <stdio.h>
#define PI 3.14
int main(void)
{
    int radius;
    float area;
    printf("Enter the radius: ");
    scanf("%d", &radius);
    area = PI*radius*radius;
    printf("Area = %f\n", area);
    return 0;
}
kali@kali:~/experiment2$ gcc -save-temps circle.c -o circle
circle.c:1:10: error: #include expects "FILENAME" or <FILENAME>
1 | #include <stdio.h>
  |          ^
circle.c:10:27: warning: missing terminating " character
10 |     printf(Area = %f\n", area);
    |                               ^
kali@kali:~/experiment2$ vi circle.c
$ gcc
gcc: fatal error: no input files
compilation terminated.
kali@kali:~/experiment2$ gcc -save-temps circle.c -o circle
kali@kali:~/experiment2$ ls
circle.c circle.i circle.o circle.s
kali@kali:~/experiment2$
```

Figure 3: Preprocessed file circle.i

4. Compilation Stage

In the compilation stage, the preprocessed C program is translated into assembly language.

EXPERIMENT 2

DEMONSTRATION OF STAGES OF COMPILE AND EXECUTION OF A C PROGRAM

The assembly output is stored in:

circle.s

A portion of the assembly code was displayed using:

head circle.s

The .s file contains assembly-language instructions generated from the C program.

5. Assembly Stage

During the assembly stage, the assembly-language program is converted into object code.

The generated object file is:

circle.o

The following command was used to identify the file:

file circle.o

The output identified circle.o as an ELF relocatable/object file.

6. Linking Stage

During linking, the object file is combined with the required libraries to create the final executable program.

The final executable created in this experiment is:

circle

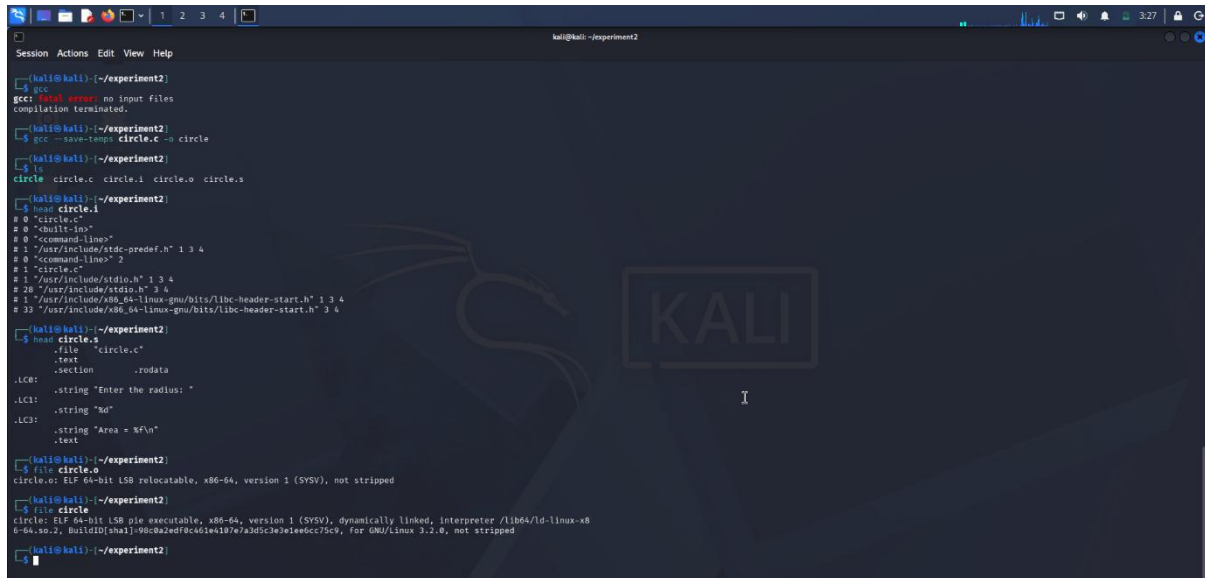
The following command was used to inspect it:

file circle

It was identified as an ELF executable file.

EXPERIMENT 2

DEMONSTRATION OF STAGES OF COMPILETIME AND EXECUTION OF A C PROGRAM



```
kali@kali: ~/experiment2
$ gcc
gcc: fatal error: no input files
compilation terminated.

$ gcc -save-temps circle.c -o circle
$ ls
circle circle.c circle.i circle.o circle.s

$ head circle.i
# 0 "circle.c"
# 0 <built-in>
# 0 <command-line>
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 <command-line> 2
# 1 "circle.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 23 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4

$ head circle.s
.file "circle.c"
.text
.section .rodata
.LC0: .string "Enter the radius: "
.LC1: .string "%d"
.LC2: .string "Area = %f\n"
.text

$ file circle.o
circle.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

$ file circle
circle: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]98c0a2edf8c40e0187e7a3d5c3e3e0ecc75c9, for GNU/Linux 3.2.0, not stripped

$
```

Figure 5: Object file and final executable file

Execution of the Program

The final executable program was executed using:

`./circle`

The program asked the user to enter the radius of the circle.

For example:

Enter the radius of the circle: 5

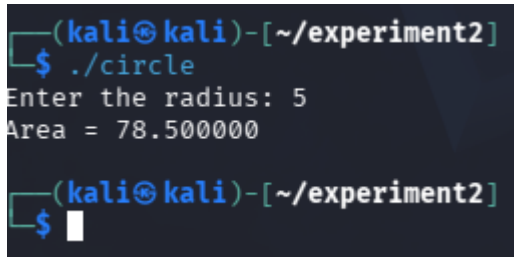
The calculated output was:

Area of the circle = 78.50

EXPERIMENT 2

DEMONSTRATION OF STAGES OF COMPILE AND EXECUTION OF A C PROGRAM

Thus, for a radius of 5 units, the calculated area of the circle was 78.50 square units

A terminal window with a dark background. The prompt is (kali㉿kali)-[~/experiment2]. The user enters ./circle. The program prompts 'Enter the radius: 5' and then outputs 'Area = 78.500000'. The prompt returns to (kali㉿kali)-[~/experiment2].

```
(kali㉿kali)-[~/experiment2]  
$ ./circle  
Enter the radius: 5  
Area = 78.500000  
  
(kali㉿kali)-[~/experiment2]  
$
```

Figure 6: Execution and output of the area-of-circle program

Compilation Flow

The complete compilation and execution sequence observed in this experiment was:

circle.c → circle.i → circle.s → circle.o → circle → Execution

Where:

.c → Source Code

.i → Preprocessed Code

.s → Assembly Code

.o → Object Code

circle → Executable Program

Result

A C program to calculate the area of a circle was successfully created, compiled, and executed in the Linux programming environment. The different stages of compilation—preprocessing, compilation, assembly, and linking—were demonstrated using the GCC --save-temps option. The intermediate .i, .s, and .o files were observed, and the final executable produced the correct area of the circle.

EXPERIMENT 2
DEMONSTRATION OF STAGES OF COMPILATION AND
EXECUTION OF A C PROGRAM

NAME – AYUSHI KUMAWAT

SAP ID – 59003252

COURSE – B.TECH CSE 2026-2030

BATCH – 16

.....

THANKYOU